

Session 7 - PGD ML

Spam Email Classification using Naive Bayes

This notebook demonstrates a simple spam email classifier using Naive Bayes, including:

- Train/Test split
- Accuracy
- Confusion Matrix
- Probability plots

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Dataset
emails = [
    "Win money now",
    "Cheap medicines available",
    "Congratulations you won a prize",
    "Call me tonight",
    "Let's have lunch tomorrow",
    "Meeting scheduled at office",
    "Free lottery tickets",
    "Earn money fast",
    "Project discussion tomorrow",
    "Are you available for meeting",
    "Limited offer win cash",
    "Reminder: office meeting today",
    "Free prize claim now",
    "Lunch at 2 pm?",
    "Earn dollars quickly"
]

labels = [
    "spam", "spam", "spam",
    "ham", "ham", "ham",
    "spam", "spam",
    "ham", "ham",
    "spam", "ham", "spam", "ham", "spam"
]

# Train/Test Split
X_train_text, X_val_text, y_train, y_val = train_test_split(
    emails, labels, test_size=0.3, random_state=42, stratify=labels
)

# Vectorization
vectorizer = CountVectorizer()
X_train = vectorizer.fit_transform(X_train_text)
X_val = vectorizer.transform(X_val_text)

# Model Training
model = MultinomialNB()
model.fit(X_train, y_train)

# Predictions
y_train_pred = model.predict(X_train)
y_val_pred = model.predict(X_val)

# Accuracy
train_acc = accuracy_score(y_train, y_train_pred)
val_acc = accuracy_score(y_val, y_val_pred)

print("Train Accuracy:", train_acc)
print("Validation Accuracy:", val_acc)

# Confusion Matrix
cm = confusion_matrix(y_val, y_val_pred, labels=["spam", "ham"])
print("\nConfusion Matrix (Validation)")
print(cm)

print("\nClassification Report")
print(classification_report(y_val, y_val_pred))
```

```

# Probabilities
classes = model.classes_
train_proba = model.predict_proba(X_train)
val_proba   = model.predict_proba(X_val)

spam_index = np.where(classes == "spam")[0][0]

train_spam_probs = train_proba[:, spam_index]
val_spam_probs   = val_proba[:, spam_index]

# Plot Train Probabilities
plt.figure()
plt.plot(range(len(train_spam_probs)), train_spam_probs, marker='o')
plt.title("Train Set: P(spam)")
plt.xlabel("Email Index")
plt.ylabel("Probability")
plt.ylim(0,1)
plt.grid(True)
plt.show()

# Plot Validation Probabilities
plt.figure()
plt.plot(range(len(val_spam_probs)), val_spam_probs, marker='o')
plt.title("Validation Set: P(spam)")
plt.xlabel("Email Index")
plt.ylabel("Probability")
plt.ylim(0,1)
plt.grid(True)
plt.show()

# Display predictions with probabilities
for text, true_label, pred_label, p_spam in zip(X_val_text, y_val, y_val_pred, val_spam_probs):
    print(f"Email: {text} | True: {true_label} | Pred: {pred_label} | P(spam)={p_spam:.3f}")

```